

mastermind manual

Draw UML class diagrams.

<https://gitlab.com/zani.manuel328/mastermind>

Version 0.2.1

Manual

1. Import	2
2. Usage	3
2.1. Parsing from source files	5
2.2. Layout	5
3. Themes	8

Documentation

4. User functions	9
4.1 class	9
4.2 row	9
4.3 column	10
4.4 group	11
4.5 draw-uml-diagram	11
4.6 draw-source-uml-diagram	11
4.7 inheritance	12
4.8 dependency	13
4.9 association	13
4.10 aggregation	14
4.11 composition	15
4.12 implementation	16

1. Import

Depending on your needs there are different types of imports:

- Import *all* the functions (recommended):

typ

```
1 #import "@preview/mastermind:0.2.1": *
```

- Import *only* the functions you want to use:

typ

```
1 #import "@preview/mastermind:0.2.1": draw-uml-diagram, class,  
    relationship, dependency, aggregation
```

- Import and (optionally) rename:

typ

```
1 #import "@preview/mastermind:0.2.1" as renamed
```

but your code would be more verbose, like in the following example:

typ

```
1 #renamed.uml-diagram({  
2   renamed.class(  
3     name: "class0",  
4     center: (0, 0)  
5   )  
6 })
```

2. Usage

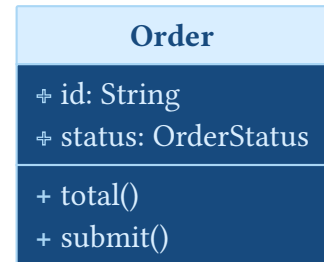
To create a class:

typ

```
1 #class(  
2   "Order",  
3   center: (0, 0),  
4   attributes: (  
5     "+ id: String",  
6     "+ status: OrderStatus"  
7   ),  
8   methods: ("+ total()", "+ submit()"),  
9 ),
```

And this draws:

```
#draw-uml-diagram(  
  (  
    class(  
      "Order",  
      center: (0, 0),  
      attributes: (  
        "+ id: String",  
        "+ status: OrderStatus"  
      ),  
      methods: ("+ total()", "+ submit()"),  
    ),  
  ),  
  (), // no relationships for now  
)
```



To create a relationship, first we need a second class:

typ

```
1 #class(  
2   "Customer",  
3   center: (0, 6),  
4   attributes: ("+ id: String", "+ email: String"),  
5   methods: ("+ placeOrder()",),  
6 ),
```

and then:

typ

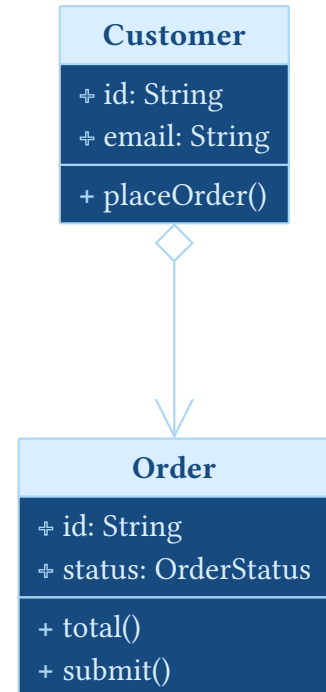
```
1 #aggregation("Customer", "Order")
```

Putting everything together:

```

#draw-uml-diagram(
  (
    class(
      "Order",
      center: (0, 0),
      attributes: (
        "+ id: String",
        "+ status: OrderStatus"
      ),
      methods: ("+ total()", "+ submit()"),
    ),
    class(
      "Customer",
      center: (0, 6),
      attributes: ("+ id: String", "+ email: String"),
      methods: ("+ placeOrder()",),
    ),
  ),
  (
    aggregation("Customer", "Order"),
  )
)

```



Arrays This package is built to work with arrays of classes and relationship – for this reason you are encouraged to use it as follows:

1. Create an array of classes
2. Create an array of relationships
3. Pass those to the `#draw-uml-diagram()` function

So the previous example code would become:

```

1 #let classes = (
2   class(
3     "Order",
4     center: (0, 0),
5     attributes: (
6       "+ id: String",
7       "+ status: OrderStatus"
8     ),
9     methods: ("+ total()", "+ submit()"),
10  ),
11  class(
12    "Customer",
13    center: (0, 6),
14    attributes: ("+ id: String", "+ email: String"),
15    methods: ("+ placeOrder()",),
16  ),
17 )
18 #let relationships = (
19   aggregation("Customer", "Order"),

```

typ

```

20 )
21 #draw-uml-diagram(classes, relationships)

```

2.1. Parsing from source files

Thanks to [sourcecraft](#), by supplying a Java or C# file, you can create the corresponding UML diagram:

From this Java file:

```

1 public abstract class Animal {
2     private String nome;
3     private int idade;
4     private Dono dono;
5
6     public String getNome() {
7         return nome;
8     }
9
10    public int getIdade() {
11        return idade;
12    }
13
14    public Dono getDono() {
15        return dono;
16    }
17
18    public abstract void emitirSom();
19 }
20

```

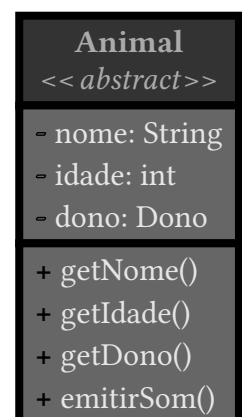
java

Its corresponding UML diagram:

```

#let sources = (
  source(
    center: (6, 0),
    "../tests/java/Animal.java"
  ),
)
#draw-source-uml-diagram(
  sources,
  themes.pixies,
  parsers.java
)

```

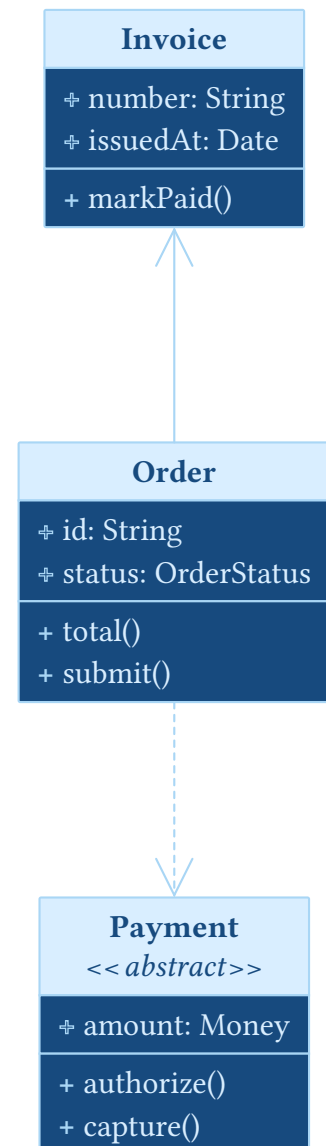


2.2. Layout

Usually it's useful to align either on a row or a column the classes – for this very reason mastermind provies the `#row()` and `#column()` functions.

Drawing in a column:

```
#let classes = (  
  ..column(  
    (0, 0), // center of the column  
    6,      // gap between the classes  
    class(  
      "Payment",  
      type: "abstract",  
      attributes: ("+ amount: Money",),  
      methods: ("+ authorize()", "+ capture()"),  
    ),  
    class(  
      "Order",  
      attributes: (  
        "+ id: String",  
        "+ status: OrderStatus"  
      ),  
      methods: ("+ total()", "+ submit()"),  
    ),  
    class(  
      "Invoice",  
      attributes: (  
        "+ number: String",  
        "+ issuedAt: Date"  
      ),  
      methods: ("+ markPaid()",),  
    ),  
  ),  
)  
#let relationships = (  
  dependency("Order", "Payment"),  
  association("Order", "Invoice"),  
)  
#draw-uml-diagram(classes, relationships)
```

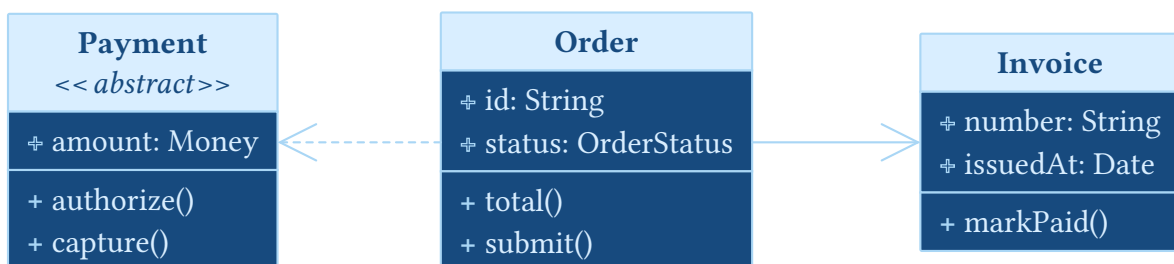


Drawing in a row:

```

#let classes = (
  ..row(
    (0, 0), // center of the row
    6,      // gap between the classes
    class(
      "Payment",
      type: "abstract",
      attributes: ("+ amount: Money",),
      methods: ("+ authorize()", "+ capture()"),
    ),
    class(
      "Order",
      attributes: (
        "+ id: String",
        "+ status: OrderStatus"
      ),
      methods: ("+ total()", "+ submit()"),
    ),
    class(
      "Invoice",
      attributes: (
        "+ number: String",
        "+ issuedAt: Date"
      ),
      methods: ("+ markPaid()",),
    ),
  ),
)
#let relationships = (
  dependency("Order", "Payment"),
  association("Order", "Invoice"),
)
#draw-uml-diagram(classes, relationships)

```



3. Themes

Available themes:

- arctic
- bauhaus
- blueprint
- botanical
- deep-ocean
- desert
- editorial
- material-design
- modern
- pixies
- terminal
- vaporwave

To use a theme, just pass it via the theme argument to the `#draw-uml-diagram()` function: typ

```
1 #draw-uml-diagram(  
2   classes,  
3   relationships,  
4   theme: themes.blueprint // this is the default  
5 )
```

To create a custom theme use the `theme()` function: typ

```
1 #let custom-theme = theme(  
2   body-fill: orange,  
3   inset: 10pt,  
4   radius: 20pt,  
5   header-fill: yellow,  
6   line-thickness: 1pt,  
7   stroke: 1pt,  
8 )
```

and pass that: typ

```
1 #draw-uml-diagram(classes,relationships, <theme: custom-theme)
```

4. User functions

For *all* the generated documentation, refer to </docs/docs.pdf>.

4.1 class

Class object.

Parameters

```
class(  
  name: string,  
  center: array,  
  type: string or none,  
  attributes: array,  
  methods: array  
) -> dictionary
```

name string

Name of the class.

center array

Center of the class.

Default: (0, 0)

type string or none

Type of the class.

Default: none

attributes array

Attributes of the class.

Default: ()

methods array

Methods of the class.

Default: ()

4.2 row

Utility to draw the given classes in a row.

Parameters

```
row(  
  center: array,  
  gap: length,  
  ..classes: class  
) -> array
```

center array

Center of the row of classes.

gap length

Gap between classes.

..classes class

The classes to draw.

4.3 column

Utility to draw the given classes in a column.

Parameters

```
column(  
  center: array,  
  gap: length,  
  ..classes: class  
) -> array
```

center array

Center of the row of classes.

gap length

Gap between classes.

..classes class

The classes to draw.

4.4 group

Utility to group the given classes.

Parameters

```
group(  
  center: array,  
  ..classes: class  
) -> array
```

center array

Center of the group of classes.

..classes class

The classes to draw.

4.5 draw-uml-diagram

Draw a UML diagram.

Parameters

```
draw-uml-diagram(  
  classes: class,  
  relationships: relationship,  
  theme: theme  
) -> content
```

classes class

The classes to draw.

relationships relationship

The relationships to draw.

theme theme

The theme of this diagram.

Default: themes.blueprint

4.6 draw-source-uml-diagram

Draw a UML diagram by parsing a source code file.

Parameters

```
draw-source-uml-diagram(  
  sources: array ,  
  theme: theme ,  
  parser: "java" "csharp"  
) -> content
```

sources array

The source files.

theme theme

The theme of this diagram.

parser "java" or "csharp"

The parser to apply.

4.7 inheritance

Create an inheritance relationship.

Parameters

```
inheritance(  
  from-rect: string ,  
  to-rect: string ,  
  from-side: "north" "east" "south" "west" ,  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect string

Ending class.

from-side "north" or "east" or "south" or "west"

Starting side.

Default: none

to-side "north" or "east" or "south" or "west"

Ending side.

Default: **none**

4.8 dependency

Create an inheritance relationship.

Parameters

```
dependency(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect string

Ending class.

from-side "north" or "east" or "south" or "west"

Starting side.

Default: **none**

to-side "north" or "east" or "south" or "west"

Ending side.

Default: **none**

4.9 association

Create an inheritance relationship.

Parameters

```
association(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect string

Ending class.

from-side "north" or "east" or "south" or "west"

Starting side.

Default: none

to-side "north" or "east" or "south" or "west"

Ending side.

Default: none

4.10 aggregation

Create an inheritance relationship.

Parameters

```
aggregation(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect `string`

Ending class.

from-side `"north" or "east" or "south" or "west"`

Starting side.

Default: `none`

to-side `"north" or "east" or "south" or "west"`

Ending side.

Default: `none`

4.11 composition

Create an inheritance relationship.

Parameters

```
composition(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect `string`

Starting class.

to-rect `string`

Ending class.

from-side `"north" or "east" or "south" or "west"`

Starting side.

Default: `none`

to-side "north" or "east" or "south" or "west"

Ending side.

Default: **none**

4.12 implementation

Create an implementation relationship.

Parameters

```
implementation(  
  from-rect: string,  
  to-rect: string,  
  from-side: "north" "east" "south" "west",  
  to-side: "north" "east" "south" "west"  
) -> relationship
```

from-rect string

Starting class.

to-rect string

Ending class.

from-side "north" or "east" or "south" or "west"

Starting side.

Default: **none**

to-side "north" or "east" or "south" or "west"

Ending side.

Default: **none**